AI-Driven Research for Systems

Audrey Cheng



UC Berkeley Sky Computing Lab

Using ML/AI to Improve Systems is Not New!

Device Plac

The Case for Learned Index Structures

daptive Video Streaming with Pensieve

Neural Packet Classific

Neo: A Learned Query Optimizer

o[2], Chi Zhang[1],
anouil[1], Nesime Tatbul[23]
abs

ABSTRACT

Packet classificatio
networking. This pr
the computation an
ticularly challengin
lutions rely on com
brittle and hard to o
In this paper, we
(RL) approach to s
There are several ch
good fit for Deep R
tively build a decis
Second, the effects o
only be evaluated c
characteristics are n
take actions that ha
it is computationall
evaluate decision tre
sample complexity

[cs.NI] 27 Feb 2019

Balsa: Learning a Query Optimizer Without Expert Demonstrations

Zongheng Yang
UC Berkeley
Berkeley, CA, USA

Wei-Lin Chiang*
UC Berkeley
Berkeley, CA, USA

Sifei Luan*
UC Berkeley
Berkeley, CA, USA

Gautam Mittal
UC Berkeley
Berkeley, CA, USA

Michael Luo
UC Berkeley
Berkeley, CA, USA

Ion Stoica
UC Berkeley
Berkeley, CA, USA

ABSTRACT

Query optimizers are a performance-critical component in every database system. Due to their complexity, optimizers take experts months to write and years to refine. In this work, we demonstrate for the first time that learning to optimize queries without learning from an expert optimizer is both possible and efficient. We present Balsa, a query optimizer built by deep reinforcement learning. Balsa first learns basic knowledge from a simple, environment-agnostic simulator, followed by safe learning in real execution. On the Join Order Benchmark, Balsa matches the performance of two expert query optimizers, both open-source and commercial, with two hours of learning, and outperforms them by up to 2.8× in workload runtime after a few more hours. Balsa thus opens the possibility of automatically learning to optimize in future compute environments where expert-designed optimizers do not exist.

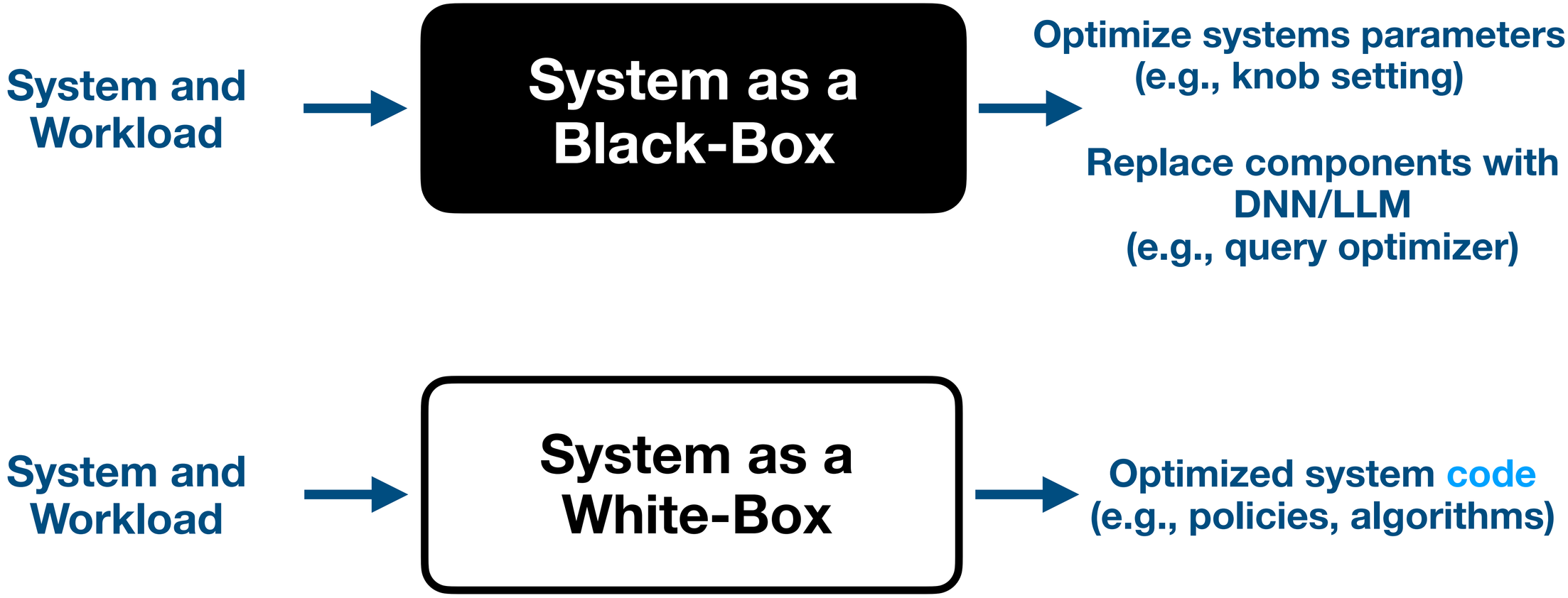
[cs.DB] 3 May 2022

As a result of this complexity, optimizers are costly to develop. Human experts may spend months to write a first version and years to refine it. For example, PostgreSQL, one of the most widely used databases in the world, has seen a continuous stream of changes to its optimizer more than 20 years after it was released [6]. Due to the high development costs, some relational systems settle for heuristic-based optimizations and postpone building full-fledged cost-based optimizers. As examples, Spark SQL was introduced in 2014 but only added a cost-based optimizer (CBO) in 2017, while CockroachDB shipped the first version of its CBO in v2.1 after "9 months of intense effort" [11].

Instead of having human experts spend years developing a state-of-the-art optimizer, in this paper we ask whether it is possible to use machine learning to *learn to optimize queries without learning from an existing expert optimizer.* We answer this question affir-

For example, DQ [22] and ReJOIN [34] use
earning combined with a human-engineered
utomatically learn search strategies to ex-
f possible join orderings. While these papers
ed search strategies can outperform conven-
s on the provided cost model, they do not
t or significant impact on actual query per-
over, they still rely on the optimizer's heuris-
lity estimation, physical operator selection,
the cost of candidate execution plan.
ches demonstrate how machine learning can
eve better cardinality estimates [20, 43, 44].
demonstrate that their improved cardinality
ually lead to better query plans. It is rel-
improve the average error of a cardinality
much harder to improve estimations for the
lly improve query plans [26]. Furthermore,
mation is only one component of an opti-
in order selection, identifying join operators
merge join) and selecting indexes cannot be

A Phase Shift: From Black-Box to White-Box



Debuggable and no runtime inference

The Disruptors



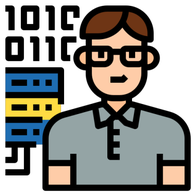
Task & SOTA Publication	Objective	SOTA / Baseline	Time / Cost
Telemetry Repair Krentsel et al. [HotNets ‘24]	Repair buggy network telemetry	+11.8% better counter repair score, +47.9% higher confidence calibration.	8h (100 iters), ≤ $15
Adaptive Weight Compression [WIP]	Assign bitrate per column to minimize bits/elem.	Similar bits/elem, 14.2% worse PPL.	12h (100 iters), ≤ $20
Cloudcast Wooders et al. [NSDI ‘24]	Optimize multi-cloud data transfer cost.	No improvement.	1h (100 iters), ≤ $15
Expert Parallelism Load Balancer [WIP]	Balance expert-parallel load across GPUs.	Same balance, 13× faster runtime vs. proprietary implementation.	5h (100 iters), ≤ $15
Global Model Placement Yu et al. (2025) [arXiv]	Optimize cost for model-to-GPU placement.	20.9% cheaper than published solution.	1h (100 iters), ≤ $15
LLM-SQL Liu et al. [MLSys ‘25]	Reorder tabular data to improve prefix hit rate.	Comparable hit rate, 3.9× faster runtime.	1h (100 iters), ≤ $20
Transaction Scheduling Cheng et al. [VLDB ‘24]	Minimize makespan in transaction scheduling.	60% better than greedy (offline).	<2h (100 iters), ≤ $20
Can’t Be Late Wu et al. [NSDI ‘24]	Schedule deadline-driven jobs on single-region spot instances.	Up to 16% (average 7%) higher cost savings vs. SOTA.	5h (100 iters), ≤ $30
Can’t Be Late Multi-Region Extension [WIP]	Schedule deadline-driven jobs on multi-region spot instances.	26% lower cost vs. single-region baseline.	3h (100 iters), ≤ $25
Sparse Attention Design Desai et al. [NeurIPS ‘25]	Balance attention sparsity and accuracy.	7% average error and density improvement vs. SOTA.	4h (100 iters), ≤ $15
Multi-Agent System Optimization Hong et al. [ICLR ‘24]	Improve multi-agent collaboration using MAST taxonomy	7% improvement on ProgramDev.	<2h (100 iters), ≤ $15
Datacenter TCP Congestion Control Addanki et al. [NSDI '22]	Maximize throughput and minimize queue latency	49% lower queue length, similar throughput.	1h (100 iters), ≤ $15

Task & SOTA Publication	Objective	SOTA / Baseline	Time / Cost
Telemetry Repair Krentsel et al. [HotNets '24]	Repair buggy network telemetry	+11.8% better counter repair score, +47.9% higher confidence calibration.	8h (100 iters), ≤ $15
Adaptive Weight Compression [WIP]	Assign bitrate per column to minimize bits/elem.	Similar bits/elem, 14.2% worse PPL.	12h (100 iters), ≤ $20
Cloudcast Wooders et al. [NSDI '24]	Optimize multi-cloud data transfer cost.	No improvement.	1h (100 iters), ≤ $15
Expert Parallelism Load Balancer [WIP]	Balance expert-parallel load across GPUs.	Same balance, 13× faster runtime vs. proprietary implementation.	5h (100 iters), ≤ $15
Global Model Placement Yu et al. (2025) [arXiv]	Optimize cost for model-to-GPU placement.	20.9% cheaper than published solution.	1h (100 iters), ≤ $15
LLM-SQL Liu et al. [MLSys '25]	Reorder tabular data to improve prefix hit rate.	Comparable hit rate, 3.9× faster runtime.	1h (100 iters), ≤ $20
Transaction Scheduling Cheng et al. [VLDB '24]	Minimize makespan in transaction scheduling.	60% better than greedy (offline).	<2h (100 iters), ≤ $20
Can't Be Late Wu et al. [NSDI '24]	Schedule deadline-driven jobs on single-region spot instances.	Up to 16% (average 7%) higher cost savings vs. SOTA.	5h (100 iters), ≤ $30
Can't Be Late Multi-Region Extension [WIP]	Schedule deadline-driven jobs on multi-region spot instances.	26% lower cost vs. single-region baseline.	3h (100 iters), ≤ $25
Sparse Attention Design Desai et al. [NeurIPS '25]	Balance attention sparsity and accuracy.	7% average error and density improvement vs. SOTA.	4h (100 iters), ≤ $15
Multi-Agent System Optimization Hong et al. [ICLR '24]	Improve multi-agent collaboration using MAST taxonomy	7% improvement on ProgramDev.	<2h (100 iters), ≤ $15
Datacenter TCP Congestion Control Addanki et al. [NSDI '22]	Maximize throughput and minimize queue latency	49% lower queue length, similar throughput.	1h (100 iters), ≤ $15

Task & SOTA Publication	Objective	SOTA / Baseline	Time / Cost
Telemetry Repair Krentsel et al. [HotNets '24]	Repair buggy network telemetry	+11.8% better counter repair score, +47.9% higher confidence calibration.	8h (100 iters), ≤ $15
Adaptive Weight Compression [WIP]	Assign bitrate per column to minimize bits/elem.	Similar bits/elem, 14.2% worse PPL.	12h (100 iters), ≤ $20
Cloudcast Wooders et al. [NSDI '24]	Optimize multi-cloud data transfer cost.	No improvement.	1h (100 iters), ≤ $15
Expert Parallelism Load Balancer [WIP]	Balance expert-parallel load across GPUs.	Same balance, 13× faster runtime vs. proprietary implementation.	5h (100 iters), ≤ $15
Global Model Placement Yu et al. (2025) [arXiv]	Optimize cost for model-to-GPU placement.	20.9% cheaper than published solution.	1h (100 iters), ≤ $15
LLM-SQL Liu et al. [MLSys '25]	Reorder tabular data to improve prefix hit rate.	Comparable hit rate, 3.9× faster runtime.	1h (100 iters), ≤ $20
Transaction Scheduling Cheng et al. [VLDB '24]	Minimize makespan in transaction scheduling.	60% better than greedy (offline).	<2h (100 iters), ≤ $20
Can't Be Late Wu et al. [NSDI '24]	Schedule deadline-driven jobs on single-region spot instances.	Up to 16% (average 7%) higher cost savings vs. SOTA.	5h (100 iters), ≤ $30
Can't Be Late Multi-Region Extension [WIP]	Schedule deadline-driven jobs on multi-region spot instances.	26% lower cost vs. single-region baseline.	3h (100 iters), ≤ $25
Sparse Attention Design Desai et al. [NeurIPS '25]	Balance attention sparsity and accuracy.	7% average error and density improvement vs. SOTA.	4h (100 iters), ≤ $15
Multi-Agent System Optimization Hong et al. [ICLR '24]	Improve multi-agent collaboration using MAST taxonomy	7% improvement on ProgramDev.	<2h (100 iters), ≤ $15
Datacenter TCP Congestion Control Addanki et al. [NSDI '22]	Maximize throughput and minimize queue latency	49% lower queue length, similar throughput.	1h (100 iters), ≤ $15

Task & SOTA Publication	Objective	SOTA / Baseline	Time / Cost
Telemetry Repair Krentsel et al. [HotNets '24]	Repair buggy network telemetry	+11.8% better counter repair score, +47.9% higher confidence calibration.	8h (100 iters), ≤ $15
Adaptive Weight Compression [WIP]	Assign bitrate per column to minimize bits/elem.	Similar bits/elem, 14.2% worse PPL.	12h (100 iters), ≤ $20
Cloudcast Wooders et al. [NSDI '24]	Optimize multi-cloud data transfer cost.	No improvement.	1h (100 iters), ≤ $15
Expert Parallelism Load Balancer [WIP]	Balance expert-parallel load across GPUs.	Same balance, 13× faster runtime vs. proprietary implementation.	5h (100 iters), ≤ $15
Global Model Placement Yu et al. (2025) [arXiv]	Optimize cost for model-to-GPU placement.	20.9% cheaper than published solution.	1h (100 iters), ≤ $15
LLM-SQL Liu et al. [MLSys '25]	Reorder tabular data to improve prefix hit rate.	Comparable hit rate, 3.9× faster runtime.	1h (100 iters), ≤ $20
Transaction Scheduling Cheng et al. [VLDB '24]	Minimize makespan in transaction scheduling.	60% better than greedy (offline).	<2h (100 iters), ≤ $20
Can't Be Late Wu et al. [NSDI '24]	Schedule deadline-driven jobs on single-region spot instances.	Up to 16% (average 7%) higher cost savings vs. SOTA.	5h (100 iters), ≤ $30
Can't Be Late Multi-Region Extension [WIP]	Schedule deadline-driven jobs on multi-region spot instances.	26% lower cost vs. single-region baseline.	3h (100 iters), ≤ $25
Sparse Attention Design Desai et al. [NeurIPS '25]	Balance attention sparsity and accuracy.	7% average error and density improvement vs. SOTA.	4h (100 iters), ≤ $15
Multi-Agent System Optimization Hong et al. [ICLR '24]	Improve multi-agent collaboration using MAST taxonomy	7% improvement on ProgramDev.	<2h (100 iters), ≤ $15
Datacenter TCP Congestion Control Addanki et al. [NSDI '22]	Maximize throughput and minimize queue latency	49% lower queue length, similar throughput.	1h (100 iters), ≤ $15

Systems Research Process



Researcher

Problem Formulation
Literature review, problem setup, goal, constraints

Evaluation Framework
Develop the system or simulator, integrate workloads and baselines

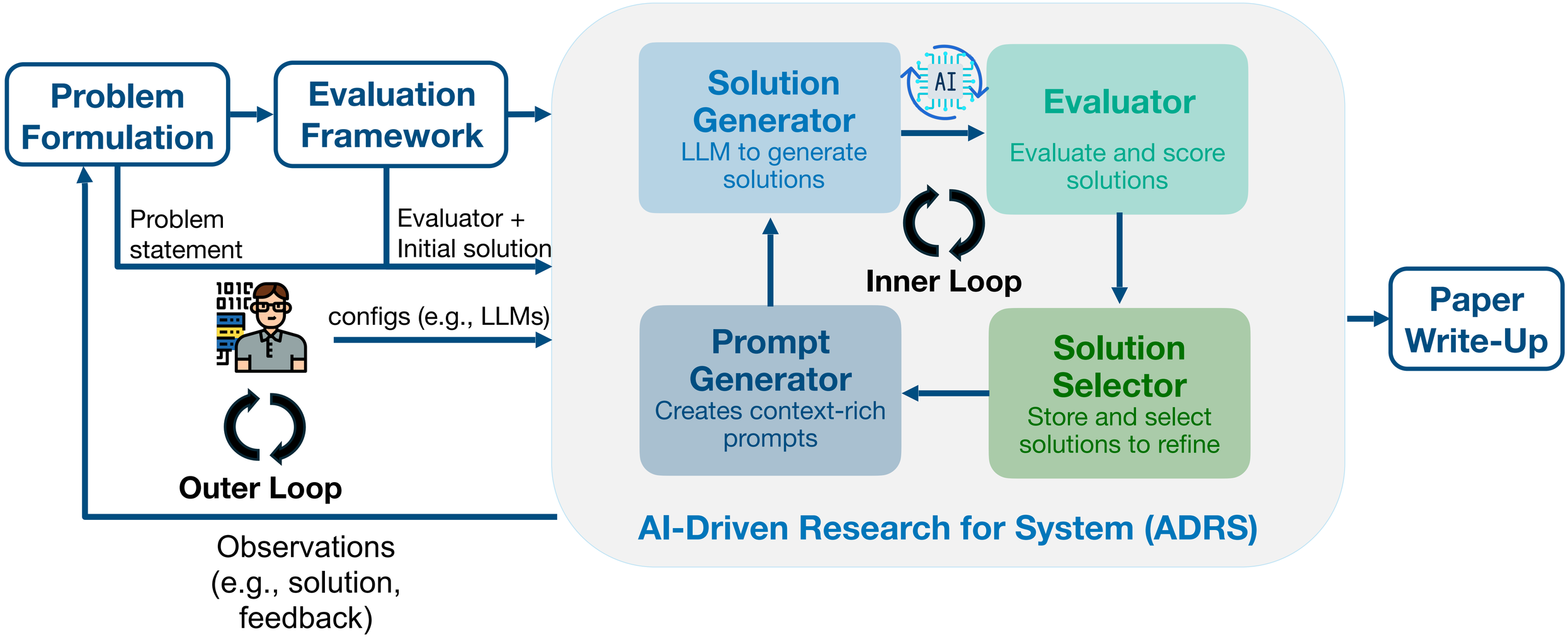
Solution
Design new solution to solve the problem

Evaluation
Leverage the evaluation framework

Paper Write-Up
Communicate findings in research paper

Experimentation Loop

AI-Driven Research for Systems (ADRS)



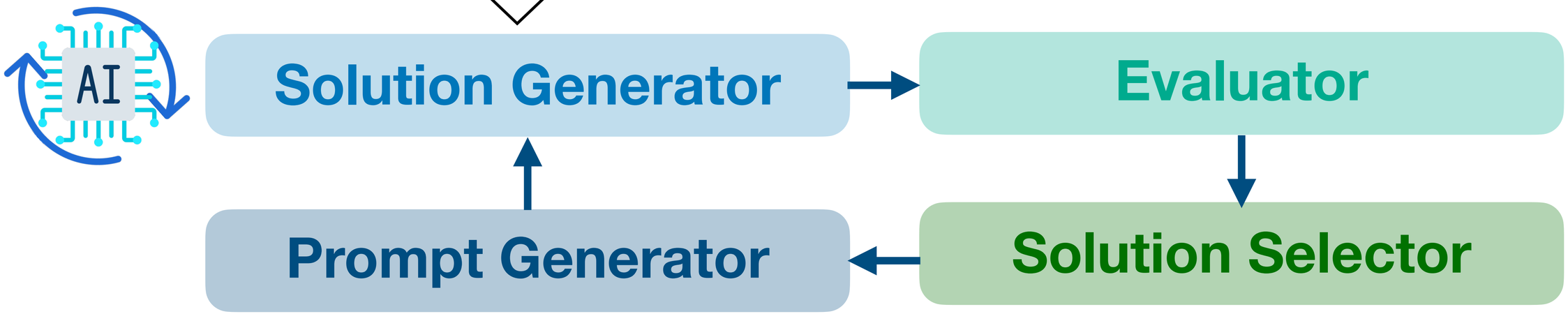
Cheng et al. Barbarian at the Gate: How AI is Upending Systems Research. arXiv (2025).

Example: Applying ADRS to Txn Scheduling

Prompt configuration

prompt:
system_message: You are an expert in database transaction processing. Your task is to improve the transaction scheduling algorithm to minimize makespan.

This algorithm will order transactions to reduce conflicts while maintaining serializability.



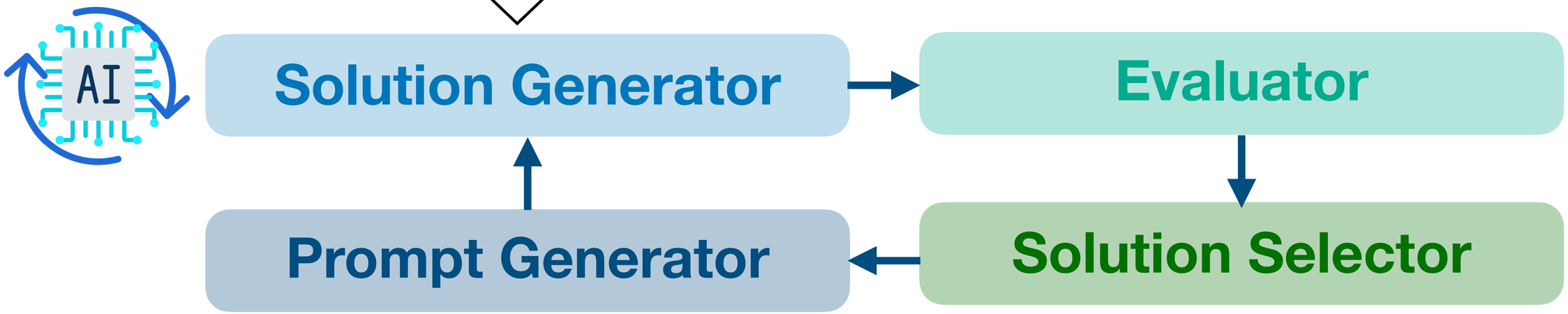
Example: Applying ADRS to Txn Scheduling

```
# This module implements the core transaction scheduler.

# EVOLVE-BLOCK-START

def get_best_schedule(workload):
    ...

# EVOLVE-BLOCK-END
```



Example: Applying ADRS to Txn Scheduling

LLM generates code between

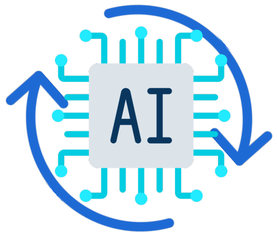
EVOLVE-BLOCK-START

<New scheduling algorithm>

EVOLVE-BLOCK-END

to implement new solution

Run simulator or full system on given workloads and compute makespan



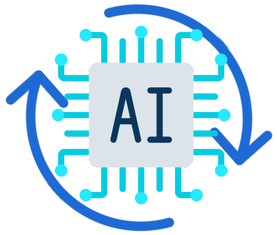
Solution Generator

Evaluator

Prompt Generator

Solution Selector

Example: Applying ADRS to Txn Scheduling



Solution Generator → **Evaluator**

↓

Solution Selector → **Prompt Generator** → (Solution Generator)

Store scheduling algorithm and corresponding makespan

Choose promising solutions to refine (e.g., with evolutionary algorithm)

Append selected solutions to prompt to provide better context for next iteration

Results

A novel policy for the offline setting

Three key innovations:

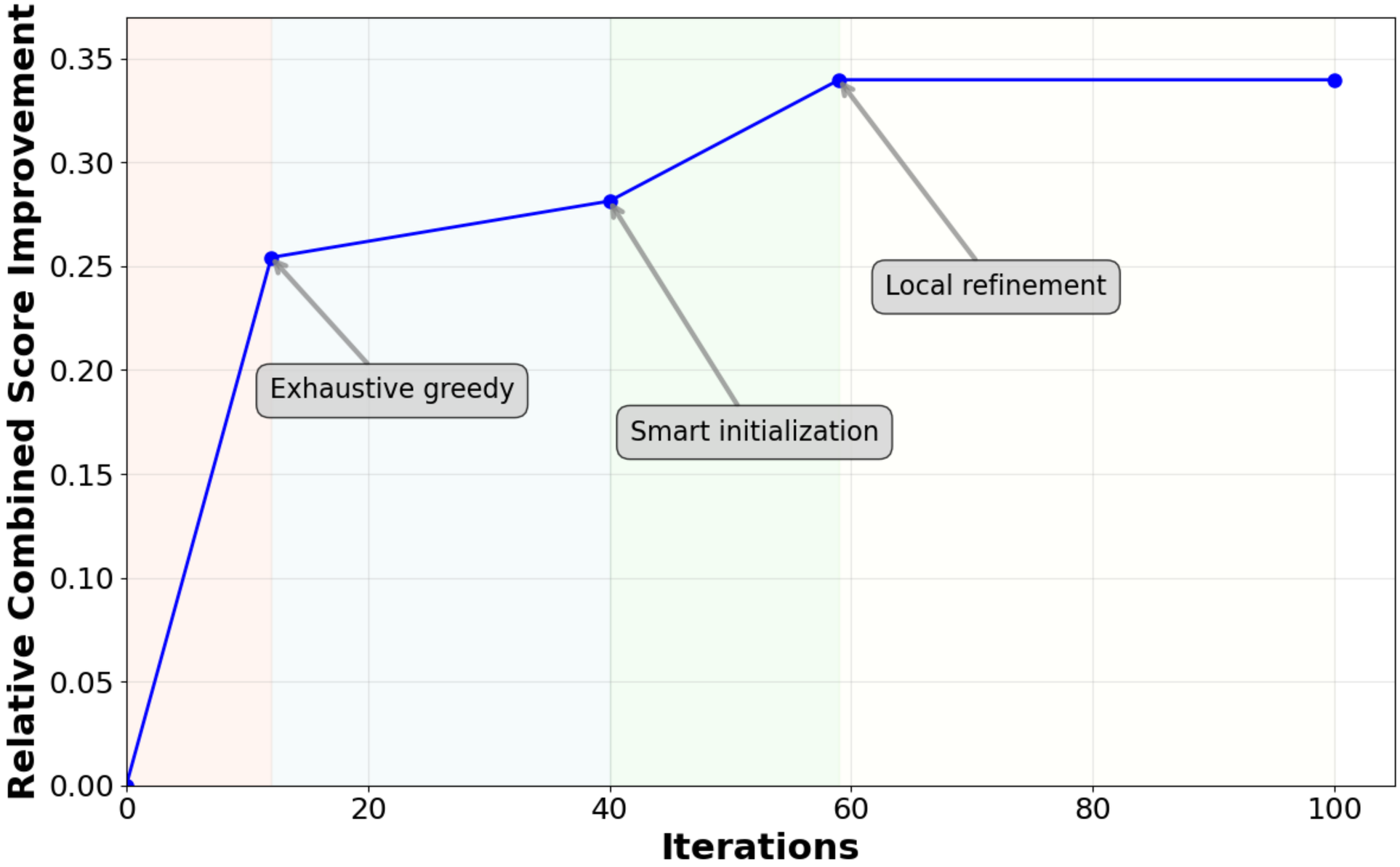
1. **Smart initialization**: sorts based on conflict heuristics
2. **Exhaustive greedy construction:** every possible position considered for each transaction
3. **Local refinement**: random pair-swap and other techniques for diversification

Baseline: SMF [*VLDB '24*]

60% lower makespan across all five benchmarks

Time: < 2hr (100 iterations)
Cost: < $20 (using GPT-5)

Evolution Process



Why Does ADRS Work?

Correctness is Easier for System Perf. Solutions

Typically, solutions do not change the systems semantics

ADRS Does Not Stop Until the Budget Expires!

Researchers often stop once with a “good enough” result

Solutions Apply Cross-Domain Techniques

LLMs ingest all existing literature, while people tend to specialize

Cross-Domain Techniques

LLMs apply concepts from a range of fields

Problem	Technique	Domain Origin	How it helps?
EPLB (GEPA, GPT 5)	**Hamilton's Apportionment**	**Political Science** (US Congress History)	**Fast-Path Allocation:** Provides a baseline for assigning integer replica counts using the "Largest Remainder Method." It's used for uniform workloads to avoid expensive search iterations.
Transaction Scheduling (OpenEvoleve, GPT-5)	**Condorcet / Borda Count**	**Social Choice Theory** (Voting Systems)	**Preference Aggregation:** Constructs a matrix where every transaction "votes" on its precedence. The score is the aggregated cost savings of essentially running an election to determine the "consensus" order.
Telemetry Repair (OpenEvolve, Gemini-3)	**Kirchhoff's Current Law** (KCL)	**Electrical Engineering** (Fluid Dynamics)	**Constraint Satisfaction:** Enforces the physical law of "Flow Conservation" to validate noisy telemetry counters. Deviations indicate data errors.

Some Lessons Learned

Less is More and More is Less

Weak baselines can lead to better solutions

Strong baseline

Might be in a local minima that is hard to get out of



Weak baseline

More freedom to find better solutions

Strategy: use different baselines

Less is More and More is Less

Fewer hints can lead to better solutions

More hints

Restrict search space and can lead to faster evolution



Fewer hints

More freedom to find better solutions

Strategy: use hints with care

Multi-stage search:

1. Start with few hints to maximize exploration
2. When stuck, try to provide more hints to direct search

Less is More and More is Less

Restricting access to high-level library APIs can lead to better solutions

High Level APIs

Faster evolution



Restricting APIs

May discover better optimizations

Strategy: provide the right level of abstraction

For example, providing entire simulator code leads to LLMs wasting time improving irrelevant code

The Solutions is Only As Good As The Evaluator

A flawed evaluator is the primary cause of flawed solutions

Prevent Overfitting

Use diverse workloads

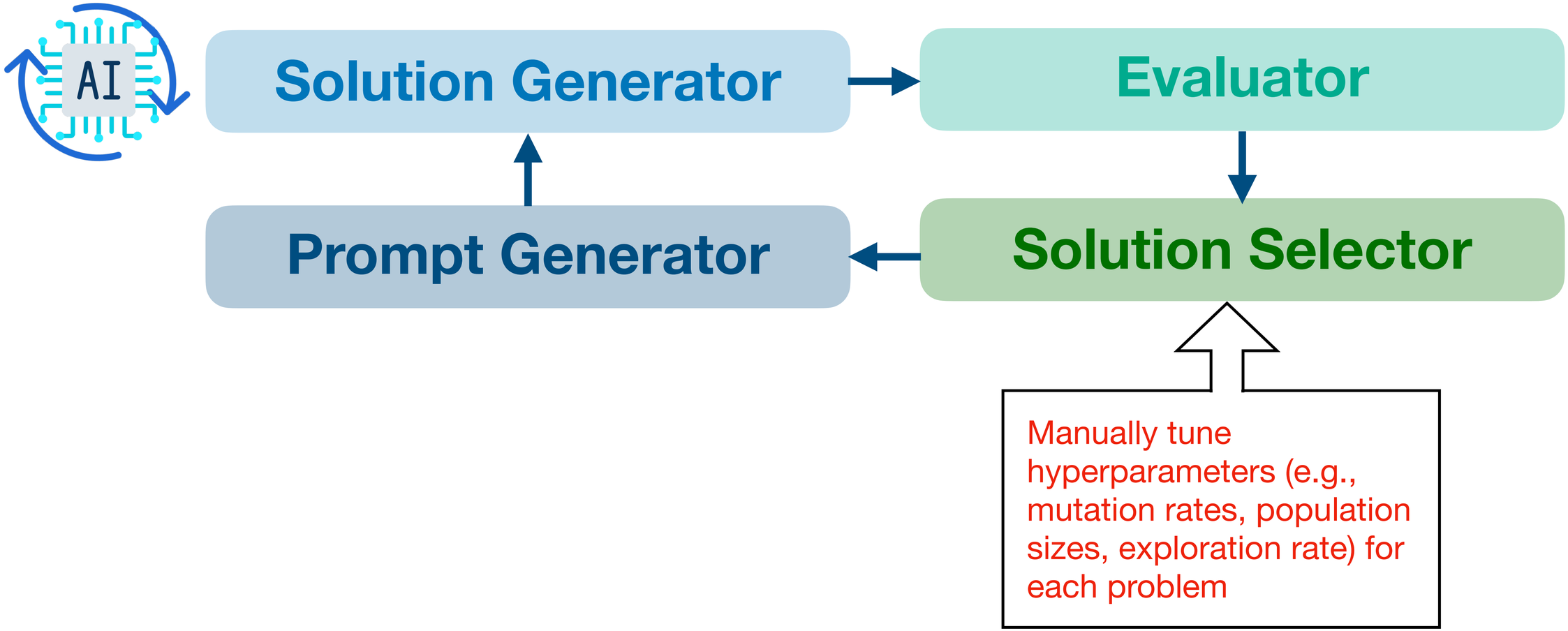
Use holdout workloads for testing

Prevent Reward Hacking

Limit scope of code modifications

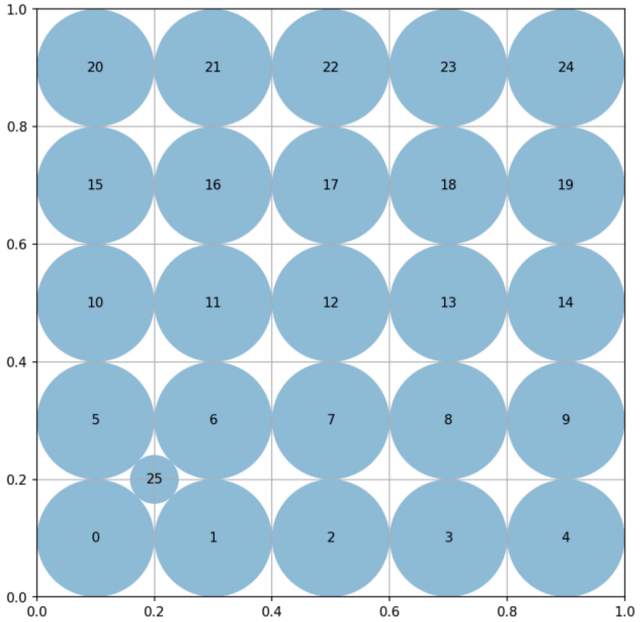
The Search is Still a Manual Process

How can we make these frameworks more effective?



A Critical Disparity in Current Frameworks

While the LLM is sophisticated and context-aware,
the search algorithms are simple and static



Circle Packing

OpenEvolve fails to converge
unless we manually:
1. Stop the run after 100 iterations
2. Restart it with a more detailed config

Search algorithm cannot adapt its behavior based on progress

AI-Driven Research for Databases

Challenges in Applying ADRS to Databases

Uniques difficulties as databases must be performant and correct

Specification

How can we determine the right goals and tradeoffs to make?

Evaluation

How can we quickly and accurately evaluate new solutions?

Verification

How can we ensure generated solutions and systems are correct?

Applying ADRS to Database Performance

Targeting well-studied, classic problems

Buffer Management

Optimize eviction algorithm to reduce disk I/O

Index Selection

Improve selection algorithm to provide better latency

Query Rewrite

Optimize policy to select rewrite rules to improve latency

Cheng et al. AI-Driven Research for Databases. Under submission.

Challenges in Evaluation

No one-size-fits-all approach

Evaluation Overheads Too High

Balance between evaluation speed and accuracy

Buffer management

E2E Performance Model Difficult to Construct

Dynamically gather problem-specific context

Index selection

Search Space Too Large

Prune search space carefully

Query rewrite

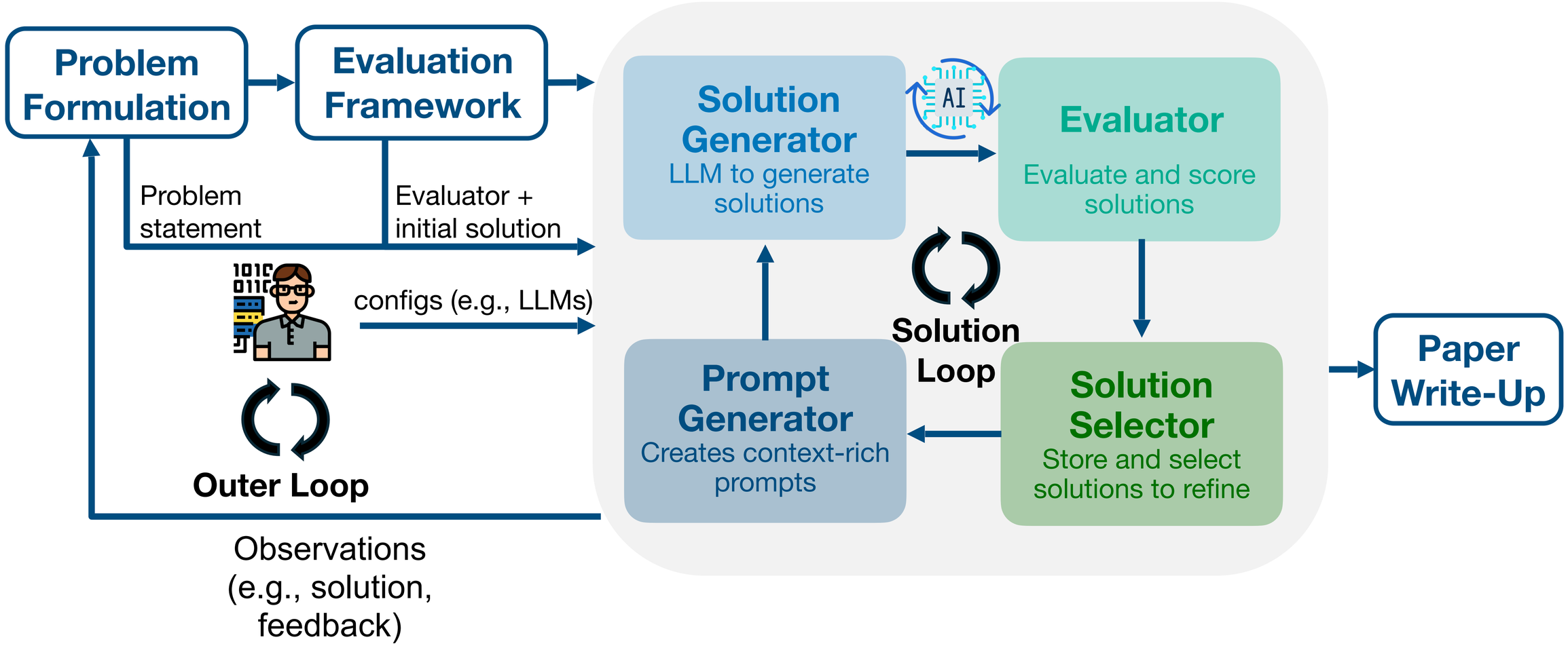
Evaluator is Well-Suited for Evolution

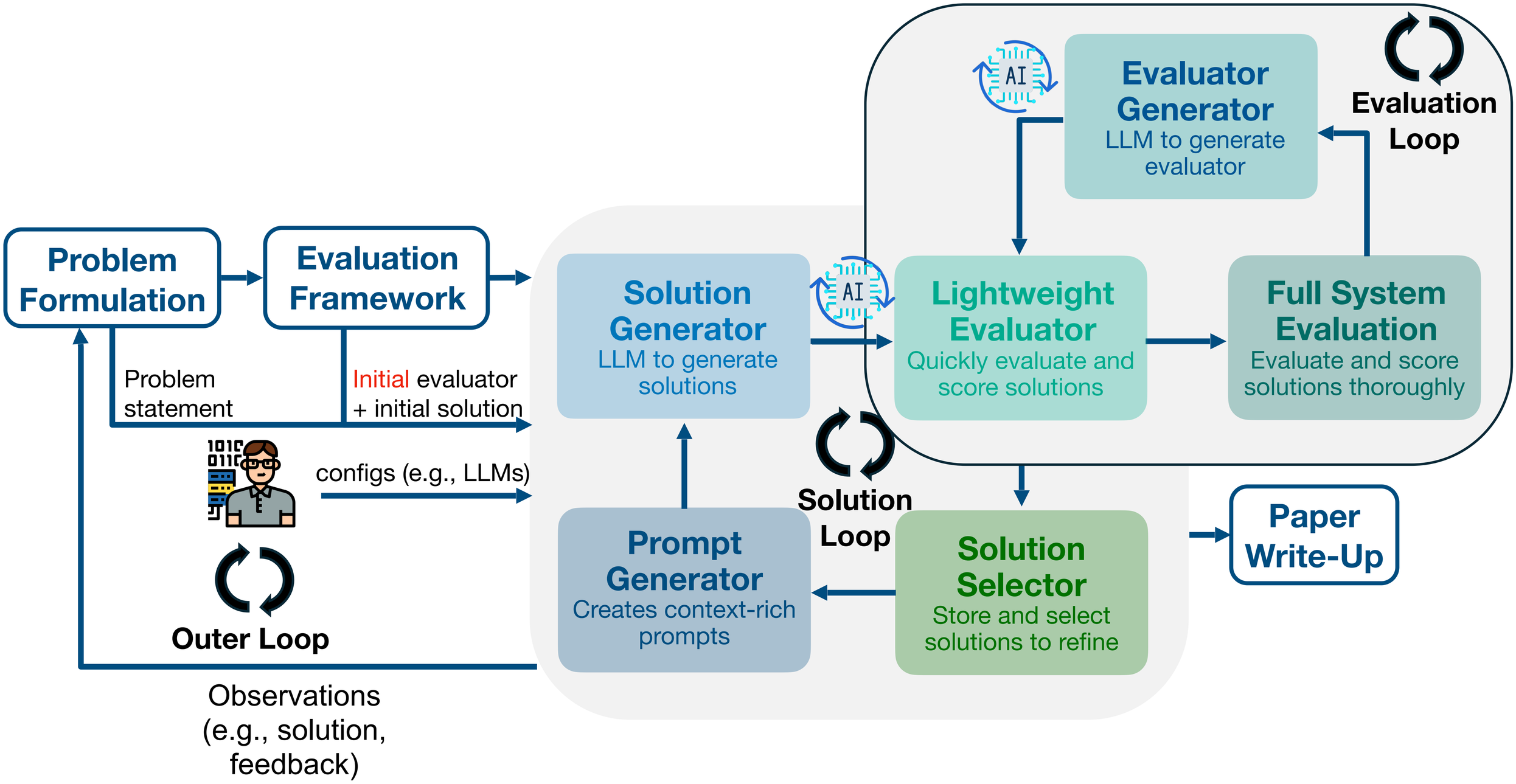
Evaluator design is an iterative process

Evaluation Techniques

Simulation, performance modeling, emulation, microbenchmarks, etc.

Key insight: co-evolve the evaluator alongside the solution

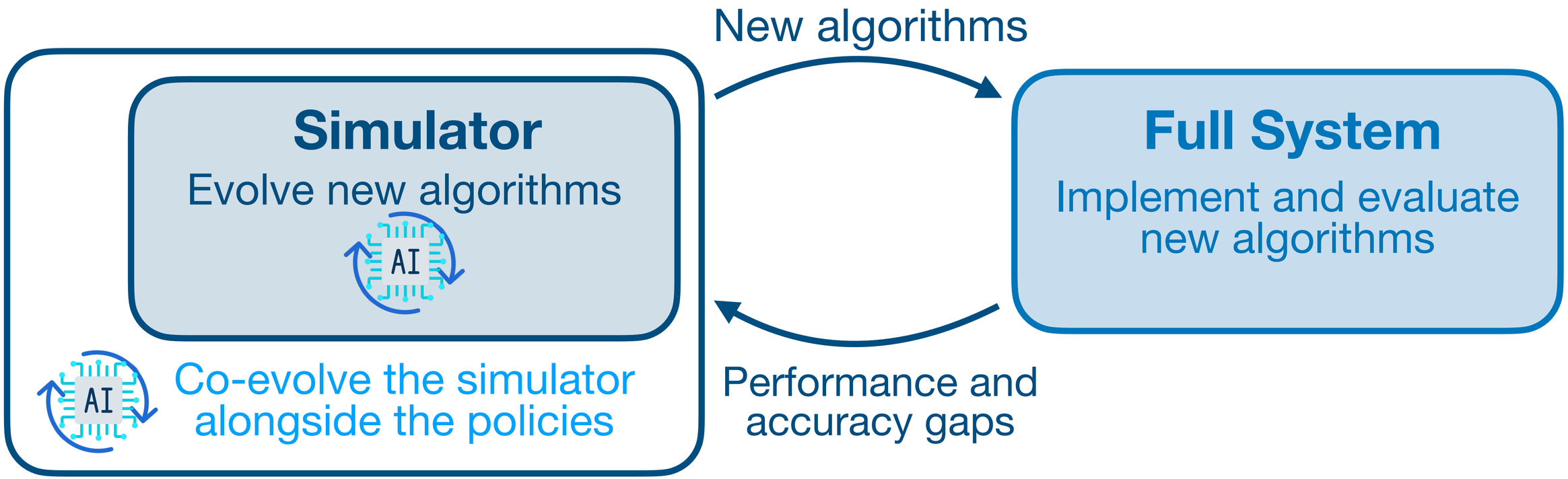

Problem Formulation
Evaluation Framework
Problem statement
Evaluator + initial solution
configs (e.g., LLMs)
Outer Loop
Observations (e.g., solution, feedback)
Solution Generator
LLM to generate solutions
AI
Evaluator
Evaluate and score solutions
Solution Loop
Prompt Generator
Creates context-rich prompts
Solution Selector
Store and select solutions to refine
Paper Write-Up


Problem Formulation
Evaluation Framework
Problem statement
Initial evaluator + initial solution
configs (e.g., LLMs)
Outer Loop
Observations (e.g., solution, feedback)
Solution Generator
LLM to generate solutions
AI
Evaluator Generator
LLM to generate evaluator
Evaluation Loop
Lightweight Evaluator
Quickly evaluate and score solutions
Full System Evaluation
Evaluate and score solutions thoroughly
Solution Loop
Prompt Generator
Creates context-rich prompts
Solution Selector
Store and select solutions to refine
Paper Write-Up

Buffer Management Optimization

Goal: evolve eviction algorithm to reduce disk I/O

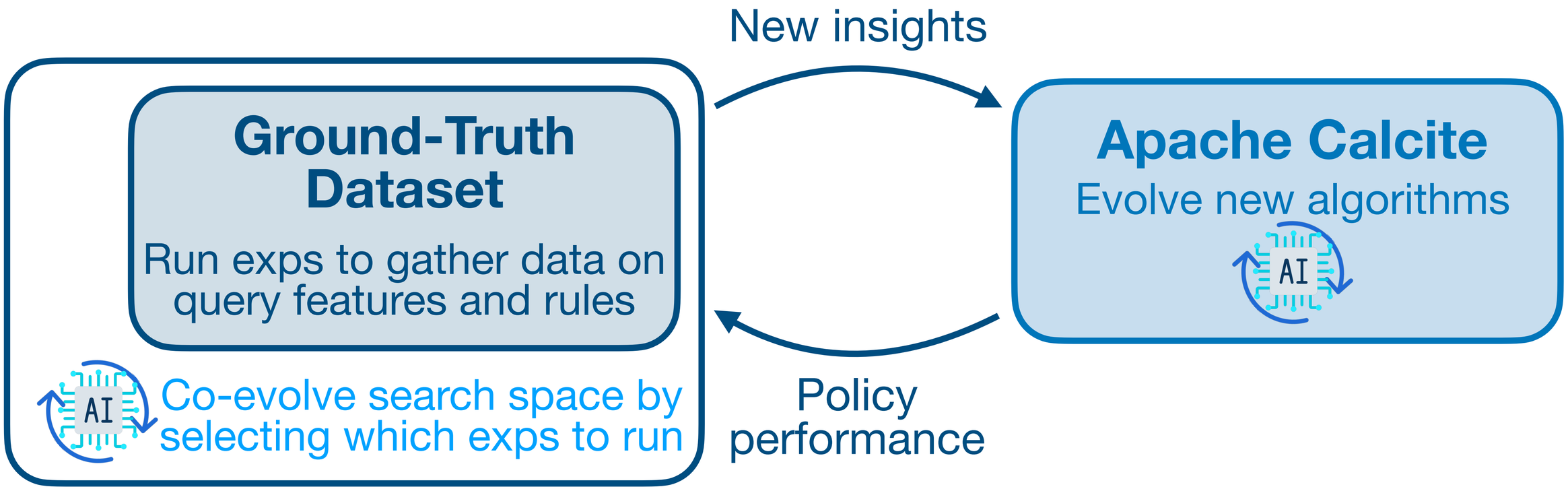
Problem: evolving on the full system (Postgres) takes too long



Query Rewrite Optimization

Goal: evolve policy to select rewrite rules to improve query latency

Problem: combinatorial search space is too large



Significant Improvements in Performance

Advance SOTA on three significant problems

Buffer Management

20% higher throughput compared to [*VLDB '26*] SOTA

Index Selection

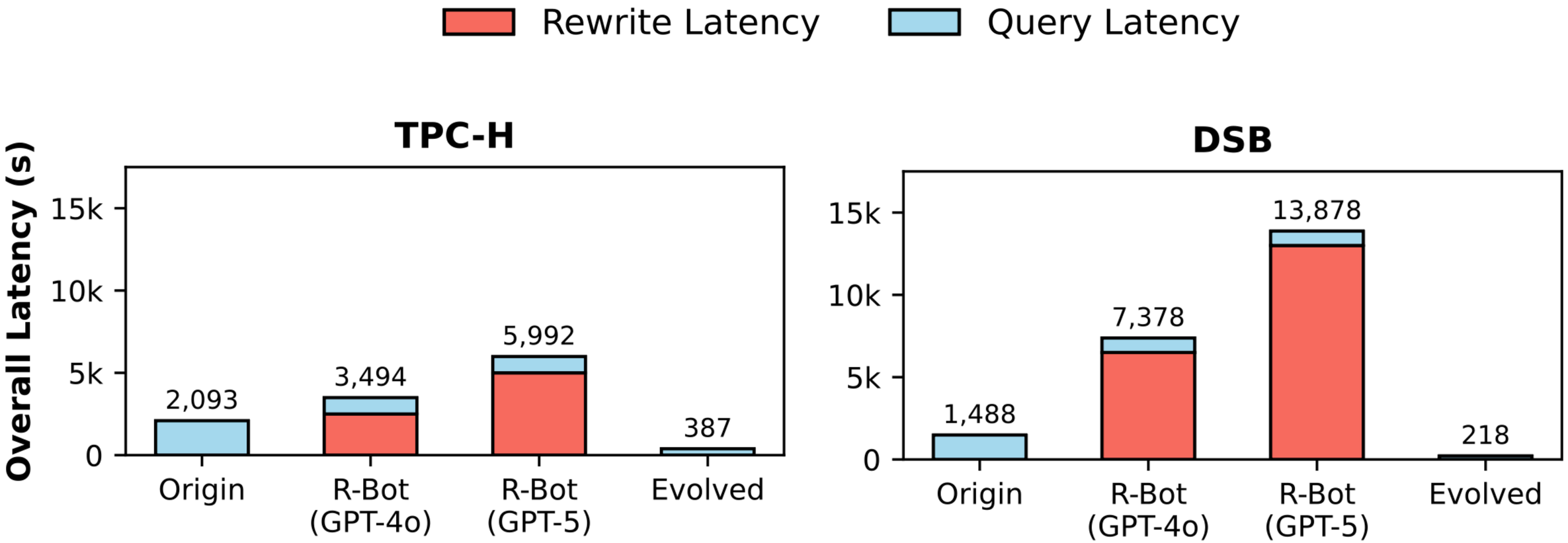
30% lower latency compared to [*VLDB '24*] SOTA

Query Rewrite

2x lower latency compared to [*VLDB '25*] SOTA

Query Rewrite Results

Compared to SOTA R-Bot [*VLDB '25*]



AI-Driven Research Is in Its Early Days

Many areas stand to benefit from this approach

ADRS

AI-driven techniques are effective for systems research

AI-Driven Research for DBs

Databases present unique challenges for ADRS

accheng@berkeley.edu
audreyccheng.com